LeetCode Practice

2 Hard: Graphs

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

2
HARD

Patterns (in order)

1. Graphs (2)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

2 questions | tap a title to jump to it

Hard questions (2) Go >

Graphs (2) Go >

H 1. Number of Islands II LC 305

H 2. String Transforms Into Another String LC 1153

Hard questions (2)

HARD | TOPIC 1 OF 1

Graphs

2 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Number of Islands II LC 305
2. String Transforms Into Another String LC 1153

HARD | Graphs | LeetCode #305 | 1 of 2 | 1/2

Number of Islands II

QUESTION

You are given an empty 2D binary grid grid of size m x n. The grid represents a map where 0's represent water and 1's represent land. Initially, all the cells of grid are water cells (i.e., all the cells are 0's).

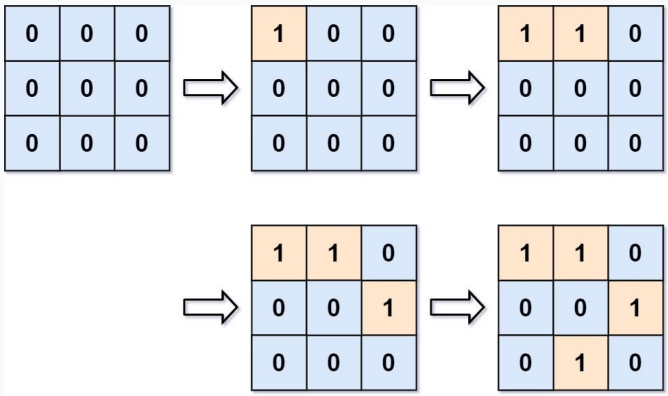
We may perform an add land operation which turns the water at position into a land. You are given an array positions where positions[i] = [r_i, c_i] is the position (r_i, c_i) at which we should operate the ith operation.

Return an array of integers answer where answer[i] is the number of islands after turning the cell (r_i, c_i) into a land.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1

Input	`m = 3, n = 3, positions = [[0,0],[0,1],[1,2],[2,1]]`
Output	`[1,1,2,3]`
Explanation	*Initially, the 2d grid is filled with water.* *- Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land. We have 1 island.* *- Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land. We still have 1 island.* *- Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land. We have 2 islands.* *- Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land. We have 3 islands.*



Example 2

Input	`m = 1, n = 1, positions = [[0,0]]`
Output	`[1]`

CONSTRAINTS

- 1 <= m, n, positions.length <= 10^4
- 1 <= m * n <= 10^4

QUESTION

- positions[i].length == 2
- 0 <= r_i < m
- 0 <= c_i < n
- Follow up: Could you solve it in time complexity O(k log(mn)), where k == positions.length?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Start with empty grid. Add land cells one at a time. After each add, return island count.

O(k log mn) per operation. Right?"

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
after every addition:
    set the cell to land and run a full
        flood-fill count of the islands
```

TIME	SPACE
O(A * m * n)	O(m * n)

WHY IT IS SLOW

Each of the A additions costs a full O(m * n) scan. Union-find, merging the new land with its land neighbours and tracking the component count, is about O(A) overall.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (Union-Find with dynamic updates)

*"Union-Find with dynamic land additions — already O(n * alpha(n)). Walk-through:"*

Start with an empty grid. parent dict is initialized lazily as land is added.

For each position (r, c) in positions:

Mark (r,c) as land. parent[(r,c)] = (r,c). count += 1.

For each of 4 neighbours (nr, nc):

If (nr,nc) is also land (already in parent):

If find(r,c) != find(nr,nc): <- in different components

```
union((r,c), (nr,nc)) <- merge the islands
count -= 1 <- two islands became one
```

Append count to result.

WHY count starts at +1 then -= for each merge:

Each new land cell starts as its own island (count++).

Each union of 2 different islands reduces total island count by 1.

Time: O(n * alpha(n)) per operation. Space: O(total land cells)."

PHASE 3 OPTIMIZE (Union-Find with dynamic updates)

Time: O(V+E) — visit each vertex and edge exactly once. Optimal DFS/BFS.

PHASE 4 CLEAN CODE

```
class _305_NumberIslandsII:
    def numIslands2(self, m: int, n: int, positions: List[
List[int]]) -> List[int]:
        parent = {}
        count  = [0]

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        def union(a, b):
            ra, rb = find(a), find(b)
            if ra == rb: return
            parent[ra] = rb
            count[0] -= 1

        result = []
        for r, c in positions:
            if (r, c) in parent:
                result.append(count[0])
                continue
            parent[(r, c)] = (r, c)
            count[0] += 1
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if (nr, nc) in parent:
                    union((r, c), (nr, nc))
            result.append(count[0])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace numIslands II on m=3,n=3, positions=[(0,0),(0,1),(1,2),(2,1)]."

Union-Find of m*n cells.

Add (0,0): mark land. count=1. No neighbors are land. ans=[1].

Add (0,1): mark land. count=2. Check neighbor (0,0)=land->union(0,1). count--=1. ans=[1,1].

Add (1,2): mark land. count=2. No land neighbors. ans=[1,1,2].

Add (2,1): mark land. count=3. Check (1,1)=water. (2,2)=water. (2,0)=water. ans=[1,1,2,3].

Return [1,1,2,3] ok WHY Union-Find: efficiently merges islands when new land neighbors existing islands.

Union merges two components -> count--. If already same component: count unchanged.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k * alpha(mn)) ~ O(k)	**O(k)** Union-Find handles dynamic connectivity perfectly

The duplicate position check avoids double-counting islands.

Given more time I'd ask: what if land cells can be removed too?

PHASE 6 COMPLEXITY & FOLLOW-UP

Split-Find is hard. Offline deletion via reverse addition is a common trick.

HARD | Graphs | LeetCode #1153 | 2 of 2 | 2/2

String Transforms Into Another String

QUESTION

Given two strings str1 and str2 of the same length, determine whether you can transform str1 into str2 by doing zero or more conversions.

In one conversion you can convert all occurrences of one character in str1 to any other lowercase English character.

Return true if and only if you can transform str1 into str2.

Example 1	
Input	`str1 = "aabcc", str2 = "ccdee"`
Output	`true`
Explanation	*Convert 'c' to 'e' then 'b' to 'd' then 'a' to 'c'. Note that the order of conversions matter.*

Example 2	
Input	`str1 = "leetcode", str2 = "codeleet"`
Output	`false`
Explanation	*There is no way to transform str1 to str2.*

CONSTRAINTS

- 1 <= str1.length == str2.length <= 10^4
- str1 and str2 contain only lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Can str1 be converted to str2 by repeatedly mapping one character to another (all occurrences)?

Conversions can be chained but must be consistent. Right?"

"If str1==str2 trivially True. If str2 uses all 26 letters -> no 'buffer' character available -> False.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
queue = [str1], seen = {str1}
for string in strings:
    apply every single-letter conversion (all
        occurrences of one letter to another)
    if str2 appears:
        return True
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Searches the space of strings. A mapping exists if each letter of str1 maps to one letter of str2 consistently and, when the mapping is not a permutation, some letter is spare (str2 uses fewer than 26 distinct letters): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"The O(n) mapping + 26-char check IS already optimal."

Walk-through of the key insight:

Build mapping: str1[i] -> str2[i]. Constraint: each char maps to exactly ONE target char.

If str1[i] already in mapping and it maps to something different -> conflict -> False.

Then: the 'buffer' check. len(set(str2)) < 26 means at least one letter is unused in str2.

WHY we need a buffer: if str1='ab' and str2='ba', we need to:

Step 1: map all 'a' -> temp (a free character). Step 2: map all 'b' -> 'a'. Step 3: map all temp -> 'b'.

If all 26 letters appear in str2, there's no free temporary character -> cycle can't be broken -> False.

If str1==str2 -> True trivially (no conversions needed).

PHASE 4 CLEAN CODE

```
class _1153_StringTransforms:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True
        mapping = {}
        for c1, c2 in zip(str1, str2):
            if mapping.get(c1, c2) != c2:
                return False
            mapping[c1] = c2
        return len(set(str2)) < 26   # need at least one u
nused char as buffer
```

PHASE 5 TEST & DEBUG

"Let me trace canConvert('aab','cde')."

Check each char mapping: a->c, a->d contradicts! -> return False? Wait

s='aab', t='cde': s[0]=a->t[0]=c; s[1]=a->t[1]=d: a maps to both c and d -> False ok

Trace s='aab',t='cca': a->c, a->c ok, b->a. All consistent and no cycle (t has unique chars? No 'c' repeats).

Special case: if all 26 chars used in t and s!=t -> can't rearrange without a free char -> False

WHY check mapping + 26-chars: need spare char for cyclic permutations (A->B->A swap).

If t uses all 26 chars and s!=t -> no temp variable available -> False.

Key edge case: if t has all 26 chars and s!=t, impossible (no free swap character). O(n).

PHASE 5 TEST & DEBUG

Graph: char-to-char mapping = directed edge. component: if t uses all 26 chars, no free intermediate char for multi-step conversion. cost=0 if same char. find(mapping) consistency.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(26)** The len(set(str2)) < 26 check: if all 26 chars appear in str2,

every potential buffer letter is 'locked' to a mapping — cycles can't be broken.

Given more time I'd ask: what if conversions can only be applied once?

Topological sort on the conversion graph — acyclic -> True.

IN YOUR APP | Expose | Editorial | Community